Palm OS Programming Recipes

Sending and Receiving Notifications

Notifications are a feature of Palm OS 3.5 and higher; they are sent by the system to notify interested applications that some event has happened or is happening.

Applications can send notification as well. For information on sending notifications, see the section Sending Notifications below. **Receiving Notifications**

For efficiency, your application registers only for notifications that you want to receive. You can find a list of available notifications in *Palm OS Programmer's API Reference* or in the header file `NotifyMgr.h`.

Different applications will register for notifications under different circumstances:

- Sometimes you only want notifications when your application is running. For example, you might want to know that an expansion card has been inserted or removed so that you can update your display.
- Sometimes you always want to be notified even when the user isn't running your application. In this case you would register when you get the `sysAppLaunchCmdSystemReset` launch code, and might never unregister. (If the user deletes your application, you are automatically unregistered from all notifications.)

The following example will show how to receive notifications when the system wakes up and goes to sleep (that is, when the handheld is turned on and off). These notifications could be used to track how a handheld is used or to monitor battery consumption, for example.

To register and use notifications, you need to do the following:

- In your `PilotMain` function, add calls to the functions `RegisterForNotifications` and `HandleNotification`.
- Create the `RegisterForNotifications` function.
- Create the `HandleNotification` function.
- Test the notification code.

Add Code to PilotMain

Add the following code to your `PilotMain` function's launch code switch statement, in the same place where you determine if the application was launched with a `sysAppLaunchCmdNormalLaunch` launch code:

```
case sysAppLaunchCmdSystemReset:
case sysAppLaunchCmdSyncNotify:  // sent when our app is installed
   RegisterForNotifications();
   break;

case sysAppLaunchCmdNotify:
   HandleNotification((SysNotifyParamType *)cmdPBP);
   break;
```

With this code added to `PilotMain`, the `RegisterForNotifications` function will be called upon every system reset, as well as when your application has been installed via a HotSync process (but not necessarily when you've been installed by a debugger or when dragged into the Palm OS Emulator).

Be very careful when registering for notifications on system resets. If you have a bug in how you handle resets (like accessing global variables), you may crash the Palm OS and force another reset, creating an infinite loop. Many users won't know to do a no-notify reset so that they can delete your application.

Create the RegisterForNotifications function

Create the `RegisterForNotifications` function:

```
static void RegisterForNotifications()
{
   UInt16 cardNo;
   LocalID dbID;

   if (0 != SysCurAppDatabase(&cardNo, &dbID))
      return; // shouldn't ever fail, but just in case.

   // Tell the system we want to know when
   // the device sleeps and wakes.
   SysNotifyRegister(cardNo, dbID, sysNotifySleepNotifyEvent, NULL,
     sysNotifyNormalPriority, 0);
   SysNotifyRegister(cardNo, dbID, sysNotifyLateWakeupEvent, NULL,
     sysNotifyNormalPriority, 0);
}
```

With this `RegisterForNotification` function added, the `HandleNotification` function will be called upon every sleep and wake. Note that the fourth parameter to the `SysNotifyRegister` function allows you to pass a pointer to a callback function that will handle the notification. Passing `NULL` as the pointer to the callback function causes the `sysAppLaunchCmdNotify` launch code to be sent to your application (we had you add code to your `PilotMain` function to handle this above). In general, it is simpler to use launch codes, but if you know you are the active application or are writing a shared library you may want to use a callback function.

Create the HandleNotification Function

What you do in the `HandleNotification` function is really up to you, but for demonstration purposes we'll keep track of how long the device was asleep (for the last time it was asleep):

```
#define kWhenWeWentToSleepFtr 1  // local definition for this routine
#define kLastSleepLengthFtr   2  // read by your app somewhere else

static void HandleNotification(SysNotifyParamType *np)
{
   UInt32 timeAsleep;

   if (np->notifyType == sysNotifySleepNotifyEvent)
      FtrSet(appFileCreator, kWhenWeWentToSleepFtr, TimGetSeconds());
   else if (np->notifyType == sysNotifyLateWakeupEvent) {
```

```
        FtrGet(appFileCreator, kWhenWeWentToSleepFtr, &timeAsleep);
        FtrSet(appFileCreator, kLastSleepLengthFtr, TimGetSeconds()
               - timeAsleep);
    }
}
```

Test the Notification Code

Now to prove the notification code works, add the following code to your main form's event handler. This code will display how long the system was asleep, reading the feature that was set in the example `HandleNotification` function.

```
case frmOpenEvent: {
    char buffer[80];
    UInt32 sleepTime;

    frmP = FrmGetActiveForm();
    MainFormInit(frmP);
    FrmDrawForm(frmP);

    // Crudely draw on top of the form's contents.  (Just for a demo!)
    FtrGet(appFileCreator, kLastSleepLengthFtr, &sleepTime);
    StrPrintF(buffer, "Last sleep time: %ld seconds.", sleepTime);
    WinDrawChars(buffer, StrLen(buffer), 0, 16);

    handled = true;
    break;
}
```

Sending Notifications

It is easy to broadcast a notification to inform other applications (or your own application) that something noteworthy has occurred. If no applications have registered for the notification that you are sending, your call to the `SysNotifyBroadcast` function will return quickly, though it does obviously take a little time for the system to make that determination.

- You can send a notification immediately by using `SysNotifyBroadcast`. In this case, you'll wait until every recipient processes the notification.
- Or else you can send a deferred notification by using `SysNotifyBroadcastDeferred`. In this case, the notification will be delivered later (currently, "later" means the next time `EvtGetEvent` is called).

The following code shows how to send a notification.

```
// must be a registered creator ID; use your
 // application's or else register a new one
 // at http://www.palmos.com/dev/creatorid/
     #define myInterestingEvent 'MyEv'


SysNotifyParamType notify;

notify.notifyType     = myInterestingEvent;
// say who is sending the event (i.e. you.)
```

```
   notify.broadcaster    = appFileCreator;
   notify.handled        = false;
   // or whatever additional information you need to send
   notify.notifyDetailsP = NULL;

        // or possibly use SysNotifyBroadcastDeferred
   SysNotifyBroadcast(&notify);
```

If you call `SysNotifyBroadcastDeferred` and have a structure that `notifyDetailsP` points to, you can simplify your memory management by choosing to have `SysNotifyBroadcastDeferred` make a copy of your structure so that you don't need to worry about knowing when to free it. To do this, pass a nonzero value as the second parameter to `SysNotifyBroadcastDeferred`. Then it will allocate a new buffer with the specified size, copy that many bytes from wherever `notifyDetailsP` points to, and change the `notifyDetailsP` element to point to its own buffer. After all recipients have handled the notification, it will free that buffer. Using this method, you can immediately free the buffer you specified in `notifyDetailsP` (or even have it point to a local variable) without worrying about making sure that the pointer stays valid until everyone has been notified, or leaking memory.